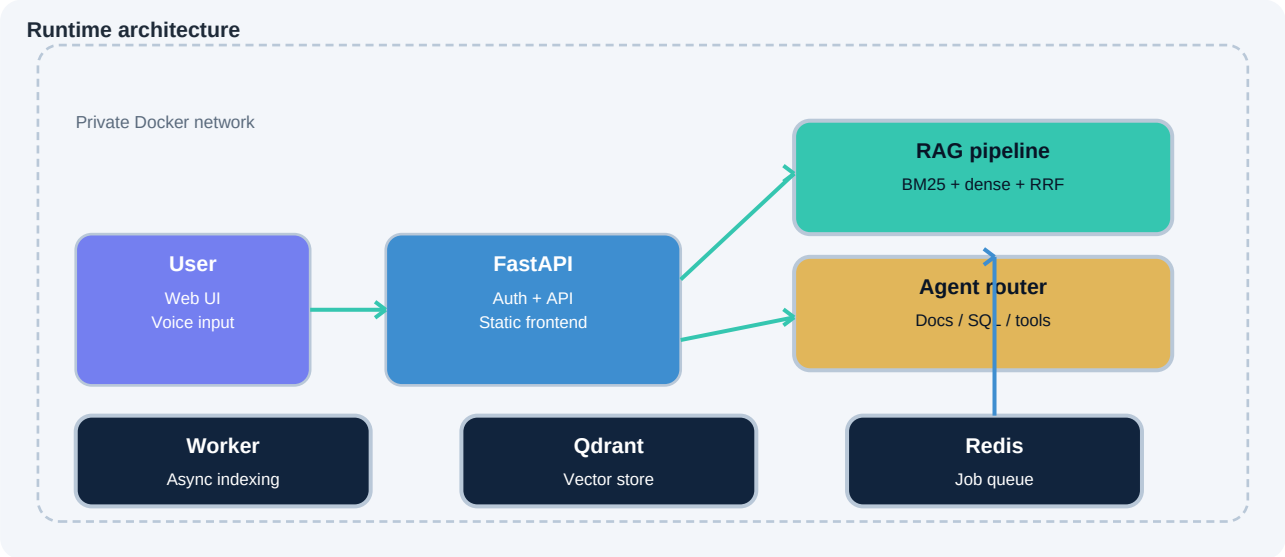
Enterprise RAG & Agentic AI Platform

Dossier technique et guide de presentation en entretien



Une plateforme de connaissance d'entreprise local-first, capable d'ingérer des documents, de retrouver des preuves, de répondre avec des citations et de router les demandes vers des outils contrôlés.

Version du dossier: 20 aout 2026 | Projet GitHub: BilalBouizdouzene/Enterprise-RAG-Agentic-AI-Platform

1. Resume executif

Ce projet simule un assistant interne d'entreprise pour les politiques RH, finance, sécurité, IT et opérations. L'objectif n'est pas seulement de produire une réponse plausible: chaque réponse documentaire doit être soutenue par des passages récupérés, cités et validés. Les demandes structurées peuvent être routées vers une base SQL en lecture seule, un outil de métadonnées ou un calculateur sécurisé.

La plateforme fonctionne localement sur CPU, avec un mode offline basé sur un embedder hashing et un provider template. Elle dispose aussi d'un profil Docker avec FastAPI, worker Redis, Qdrant, PostgreSQL et MinIO. Cette approche permet de démontrer les choix d'architecture sans dépendre d'une clé LLM ou d'un service cloud pour le parcours principal.

Ce que le projet demontre

- Conception d'un pipeline RAG complet: ingestion, nettoyage, chunking, retrieval, fusion, validation et citations.

- Architecture agentique contrôlée avec routage vers document_search, SQL read-only, metadata_search et calculator.

- Capacité produit: interface web, upload multi-format, indexation asynchrone, voice input navigateur et dashboard de qualité.

- Discipline d'ingénierie: tests pytest, Ruff, Docker Compose, health checks, audit des uploads et documentation.

Sommaire

Section	Contenu
2	Probleme metier et objectifs
3	Architecture technique
4	Pipeline RAG detaille
5	Agentic AI et outils
6	Donnees, ingestion et uploads
7	Evaluation et resultats
8	Interface et experience utilisateur
9	Execution locale et Docker
10	Securite, limites et prochaines etapes
11	Guide d'entretien
12	Questions techniques et reponses

2. Probleme metier et objectifs

Dans une entreprise, l'information utile est souvent dispersée dans des politiques, procédures, PDF, notes et bases structurées. Une recherche par mots-clés seule manque les paraphrases; un LLM seul peut halluciner. La plateforme traite ces deux risques séparément: retrieval pour trouver les preuves, génération contrôlée pour formuler la réponse.

Objectifs fonctionnels

- Permettre à un collaborateur de poser une question en langage naturel.
- Retourner une réponse lisible avec des sources [S1], [S2], etc.
- Refuser ou signaler l'incertitude lorsque le corpus ne contient pas de preuve suffisante.
- Accepter des documents PDF, TXT, Markdown, HTML, DOCX et images comme pièces jointes.
- Traiter séparément les questions documentaires, structurées et calculatoires.

Objectifs techniques

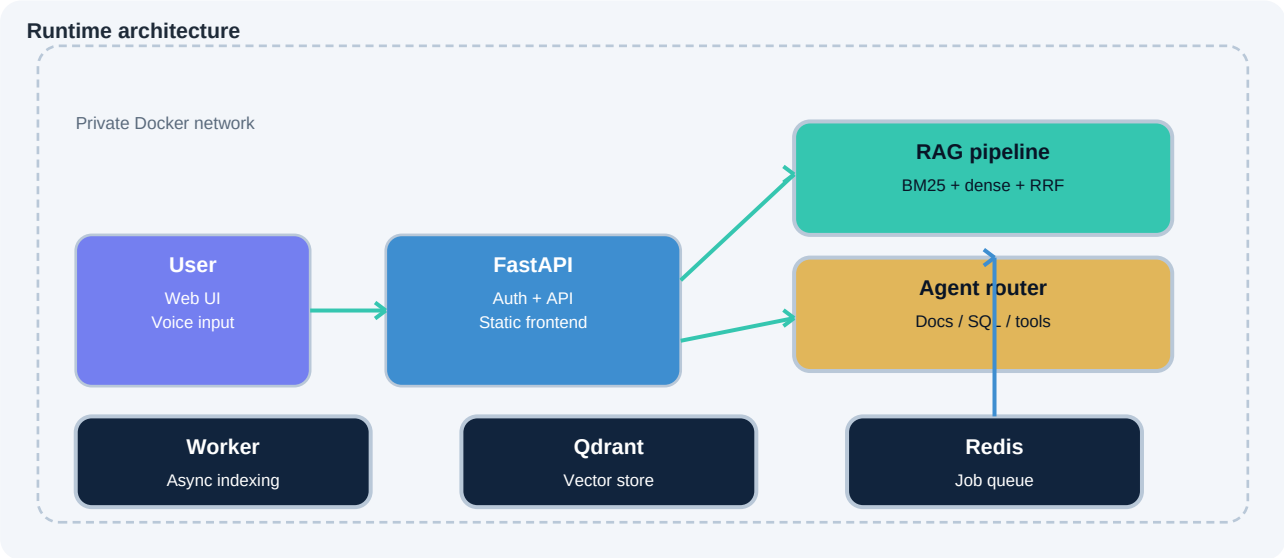
- Rester exécutable localement avec peu de dépendances et sans GPU.
- Pouvoir passer à Qdrant, Redis, MinIO et à un vrai LLM sans réécrire le coeur applicatif.
- Mesurer retrieval, RAG et routage agentique séparément.
- Préparer un dépôt portfolio lisible et reproductible.

Phrase de synthese

J'ai conçu un assistant de connaissance interne qui sépare la recherche de preuves, la génération et les outils structurés afin d'améliorer la traçabilité et de réduire les hallucinations.

3. Architecture technique

L'architecture suit une séparation claire entre expérience utilisateur, gateway API, intelligence de recherche, outils agentiques et services d'infrastructure. Le même coeur applicatif peut démarrer en mémoire pour le développement ou s'appuyer sur des services persistants dans Docker.



Composants principaux

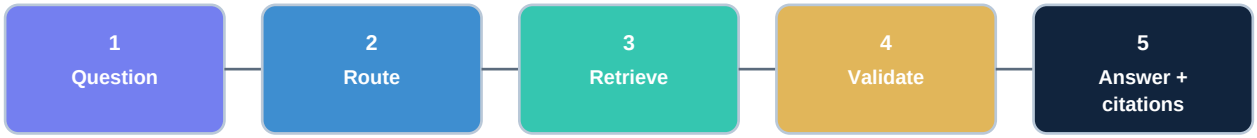
Composant	Role	Choix du projet
frontend_web	Interface chat, explorer, evaluation, system status, upload et voice.	HTML/CSS/JavaScript servi par FastAPI
FastAPI	API REST, auth API key, health checks, upload, ask, search et metrics.	Gateway Python
RAGPipeline	Orchestre retrieval, reranking, validation et génération.	Pipeline testable et modulaire
BM25 + dense	Combine recherche lexicale et similarité vectorielle.	Hashing CPU par défaut; SentenceTransformers optionnel
Qdrant	Stocke les vecteurs dans le profil Docker.	Persistance par volume Docker
Redis + worker	Découple les uploads et la réindexation.	File enterprise_rag:index_jobs
MinIO	Conserve une copie objet des originaux.	S3-compatible, optionnel en local
PostgreSQL / SQLite	Données structurées et outils SQL read-only.	SQLite local; PostgreSQL provisionné dans Docker

Flux de demarrage

- Le navigateur appelle la gateway FastAPI sur le port 8000.
- La gateway charge les chunks et construit le retriever au démarrage.
- En profil Docker, le retriever utilise Qdrant et les uploads documentaires passent par Redis vers le worker.
- Le worker relance ingestion et build_index, puis appelle /admin/reload-index avec la clé API.

4. Pipeline RAG detaille

Le pipeline RAG est conçu comme une chaîne mesurable. Chaque étape peut être évaluée séparément, ce qui permet de distinguer un problème de retrieval d'un problème de génération.



Etape	Explication
1. Ingestion	Les fichiers sont lus par format, nettoyés et convertis en documents avec identifiant, titre, source, métadonnées et page.
2. Chunking	Le texte est découpé par section ou taille contrôlée avec chevauchement. Le chunk conserve document_id, page et section pour les citations.
3. Retrieval lexical	BM25 favorise les termes exacts, les acronymes, les montants et les noms de procédures.
4. Retrieval dense	L'embedder hashing CPU fournit une représentation déterministe légère. Un backend SentenceTransformers peut être installé séparément.
5. Fusion RRF	Les classements lexical et dense sont fusionnés pour améliorer le rappel sur les formulations variées.
6. Reranking	Un reranker optionnel peut réordonner les candidats; le profil léger utilise un fallback lexical.
7. Validation	Le validateur mesure si le contexte soutient la réponse et déclenche un refus lorsque les preuves sont insuffisantes.
8. Generation	Le provider template répond offline; OpenAI-compatible ou Ollama peuvent être activés par configuration.

Pourquoi les citations sont importantes

Une réponse générée sans preuve est difficile à auditer. Ici, chaque source contient un label, un titre, une page éventuelle et un extrait. En entretien, il faut expliquer que les citations ne garantissent pas à elles seules la vérité: elles rendent la chaîne vérifiable et permettent de mesurer le support contextuel.

5. Agentic AI et outils controles

L'agent n'est pas un agent généraliste autorisé à exécuter n'importe quelle action. C'est un graphe contrôlé qui route une intention vers un petit ensemble d'outils déterministes et en lecture seule.

Route	Outil	Garde-fou
Documentaire	document_search	Réponse fondée sur les chunks et citations
Structurée	sql_query	SQL validé en lecture seule; pas d'INSERT, UPDATE ou DELETE
Métadonnées	metadata_search	Filtrage par département et catégorie
Calcul	calculator	AST limité à l'arithmétique; pas d'exécution Python arbitraire

Le graphe conserve l'état, limite les retries et expose les outils utilisés dans la réponse. Cette approche est plus facile à tester et à auditer qu'un agent ouvert qui pourrait appeler des outils non prévus.

Exemple de raisonnement attendu

Pour une question sur le nombre d'employés en France, l'agent identifie une demande structurée et utilise SQL read-only. Pour une question sur une politique de sécurité, il route vers la recherche documentaire. Pour un calcul de budget, il peut utiliser le calculateur sécurisé après avoir récupéré les valeurs nécessaires.

6. Donnees, ingestion et uploads

Le corpus de démonstration est synthétique et couvre plusieurs départements. Il permet de tester la volumétrie, les filtres, les questions multi-documents et les refus sans exposer de données d'entreprise réelles.

Type	Traitement	Etat actuel
PDF	Extraction texte + pages + chunking	Indexé si le PDF contient du texte
DOCX	Extraction paragraphes et sections	Indexé
TXT / Markdown / HTML	Nettoyage et chunking	Indexé
PNG / JPG / WebP	Stockage visuel	Conservé; OCR à activer

Upload synchrone ou asynchrone

- En local, INDEXING_MODE=sync peut réindexer immédiatement.
- Dans Docker, INDEXING_MODE=async place le document dans Redis et renvoie un job_id.
- Le worker relance le pipeline puis recharge l'index via l'API.
- Les originales peuvent être copiées dans MinIO quand S3_ENDPOINT_URL est défini.
- L'interface affiche le nombre de fichiers reçus, indexés, en attente et les erreurs.

Etat observe dans la version locale

La dernière validation a montré 639 documents et 1 926 chunks dans l'index. Le projet comporte également des tests API et ingestion. Les documents personnels ou uploadés ne doivent pas être publiés sur GitHub; ils sont exclus par .gitignore.

7. Evaluation et resultats

Le projet sépare trois niveaux d'évaluation: retrieval, RAG et agent. Cette séparation est importante pour diagnostiquer les erreurs et éviter de résumer la qualité par une seule métrique.

Niveau	Mesures	Interpretation
Retrieval	Recall@5, MRR, nDCG@5, latence	Les bonnes sources remontent-elles?
RAG	Support contextuel, refus, exactitude attendue	La réponse est-elle soutenue et prudente?
Agent	Accuracy de routage, appels outil, retries	La demande part-elle vers le bon outil?
Qualité opérationnelle	Health, upload errors, queue depth, latence	Le service reste-t-il exploitable?

Snapshot de benchmark disponible dans le dépôt

Pipeline	Recall@5	MRR	nDCG@5	Latence moyenne
BM25	0.933	0.917	0.903	8.406 ms
Dense hashing	0.507	0.451	0.424	0.470 ms
Hybrid RRF	0.731	0.679	0.653	11.139 ms
Hybrid + fallback lexical	0.782	0.798	0.771	10.563 ms

Lecture a donner en entretien

BM25 est très fort sur ce corpus car les questions partagent des termes exacts avec les politiques. Le dense hashing est une baseline CPU et non un modèle sémantique de production. La fusion hybride améliore la robustesse globale, mais ajoute une petite latence. Cette analyse montre que je mesure les compromis au lieu de présenter un score isolé comme une vérité absolue.

8. Interface et experience utilisateur

L'interface web est servie directement par FastAPI et propose quatre espaces: Knowledge Assistant, Document Explorer, Evaluation Center et System Status.

Vue	Valeur utilisateur
Knowledge Assistant	Question en langage naturel, réponse avec sources, outils utilisés et historique de session.
Document Explorer	Recherche par titre, département et catégorie; ajout multi-fichiers.
Evaluation Center	Lecture des métriques retrieval, RAG et agent issues des rapports.
System Status	Etat API, index, embedding backend, nombre de documents et chunks.

Voice input

Le bouton Voice utilise Web Speech API dans Chrome ou Edge. La transcription reste côté navigateur et peut être corrigée avant l'envoi. L'audio n'est pas envoyé au serveur dans ce profil local, ce qui réduit le périmètre de confidentialité.

Question recommandee pour une capture portfolio

> *What should I do if I lose my work laptop?*

Cette question produit une réponse courte, utile et citée à partir de la procédure de perte d'ordinateur. Pour une capture plus technique, ajoutez ensuite un PDF et montrez le statut d'indexation.

9. Execution locale et Docker

Mode local

```
cd "C:\Users\Probook\Documents\ChatGPT\Enterprise RAG & Agentic AI Platform"
.\.venv\Scripts\Activate.ps1
python -m uvicorn enterprise_rag.api.main:app --reload
```

Ouvrir ensuite http://localhost:8000. Après ajout de documents en mode synchrone, les commandes de référence sont:

```
python scripts/ingest.py
python scripts/build_index.py
```

Mode Docker

```
docker compose up -d --build
docker compose ps
docker compose logs -f api worker
```

Services Docker

Service	Fonction
api	FastAPI + interface web sur le port 8000
worker	Indexation asynchrone Redis
qdrant	Vecteurs et recherche dense persistante
redis	File de jobs
postgres	Base relationnelle provisionnée pour la persistance entreprise
minio	Stockage objet S3-compatible des originaux

Commandes de controle

- Health API: http://localhost:8000/health

- Swagger: http://localhost:8000/docs

- Arret: docker compose down

- Rebuild apres modification frontend: docker compose up -d --build api

10. Securite, limites et prochaines etapes

Protections presentes

- API key optionnelle via X-API-Key, activée dans le profil final local.

- Endpoints health/live et health/ready séparés des endpoints métier.

- Taille maximale upload de 25 MB et extensions limitées.

- Audit JSONL des uploads et erreurs.

- Qdrant, Redis et PostgreSQL non exposés directement par le profil Docker de base.

- Secrets conservés dans .env, ignoré par Git.

Limites a dire explicitement

- Le SSO/OIDC, RBAC et la rotation de secrets nécessitent un fournisseur d'identité et une configuration d'entreprise.

- L'OCR/vision des images n'est pas activé dans le profil local actuel.

- PostgreSQL est provisionné par Docker, mais certaines données structurées de démonstration utilisent encore SQLite.

- Le provider template est utile pour la démonstration offline et ne représente pas la qualité d'un LLM de production.

- Une exposition publique demande HTTPS, reverse proxy, sauvegardes, antivirus et monitoring externe.

Roadmap realiste

Phase	Action	Valeur
P0	HTTPS, secrets manager, SSO/RBAC, rate limiting, antivirus uploads	Réduire le risque avant exposition
P1	PostgreSQL pour métadonnées/audit, backups, monitoring et alertes	Exploitation durable
P2	OCR, embeddings multilingues et reranking	Meilleure couverture sémantique
P3	CI/CD cloud, autoscaling, tests de charge et gouvernance des données	Passage à l'échelle

11. Guide d'entretien

Pitch de 90 secondes

J'ai construit une plateforme de connaissance d'entreprise basée sur RAG et Agentic AI. Elle ingère des politiques PDF, DOCX, Markdown et HTML, les nettoie et les découpe en chunks avec métadonnées. Pour répondre, elle combine BM25 et retrieval dense, fusionne les candidats, valide le support contextuel et retourne des citations. Un routeur agentique peut choisir entre recherche documentaire, SQL read-only, filtres de métadonnées et calcul sécurisé. Le projet fonctionne en local avec un embedder hashing et un provider offline, puis peut passer à Docker avec Qdrant, Redis, worker, PostgreSQL et MinIO. J'ai séparé les métriques de retrieval, RAG et routage afin de mesurer précisément les compromis entre rappel, latence et fiabilité.

Version courte si le recruteur demande votre contribution

Ma contribution principale a été de concevoir les interfaces entre ingestion, retrieval, génération et outils, puis de rendre le système testable et démontrable. J'ai aussi travaillé sur les uploads, l'indexation asynchrone, les citations, l'authentification API et la containerisation Docker.

Comment expliquer les choix

Question du recruteur	Reponse conseillee
Pourquoi RAG?	Parce que les politiques changent et que les réponses doivent être ancrées dans des sources internes vérifiables.
Pourquoi BM25 + dense?	BM25 est fort sur les termes exacts; le dense aide sur les paraphrases. La fusion combine leurs forces.
Pourquoi un embedder hashing?	C'est une baseline déterministe, CPU et sans téléchargement lourd, adaptée à une démo reproductible. Je garde une voie d'évolution vers des embeddings multilingues.
Comment réduisez-vous les hallucinations?	Retrieval de candidats, validation de preuves, citations et refus lorsque le contexte ne soutient pas la réponse.
Pourquoi un agent?	Pour router des demandes hétérogènes vers des outils spécialisés, pas pour laisser un LLM exécuter des actions arbitraires.

Structure de reponse en entretien

- Contexte: quel problème métier?
- Décision: quelle architecture et pourquoi?
- Preuve: quelles métriques ou tests?
- Limite: qu'est-ce qui reste à améliorer?
- Suite: comment passeriez-vous en production?

12. Questions techniques et reponses

Que se passe-t-il si la réponse n'est pas dans les documents?

Le validateur considère le support insuffisant et le système peut refuser au lieu d'inventer. C'est un comportement attendu et mesuré par le taux de refus.

Comment gérez-vous un document ajouté?

L'API valide le format et la taille, écrit l'original dans le dossier d'upload, le copie éventuellement dans MinIO, puis indexe directement ou envoie un job Redis au worker.

Pourquoi ne pas mettre l'audio sur le serveur?

Le profil local utilise Web Speech API côté navigateur. Cela réduit la surface de données et permet un parcours vocal sans pipeline audio serveur.

Comment testeriez-vous le système?

Tests unitaires du chunking et retrieval, tests API d'auth et upload, évaluations offline avec questions answerable et non answerable, puis tests de charge et de sécurité avant production.

Quels sont les risques en production?

Secrets mal gérés, documents malveillants, absence de contrôle d'accès fin, dérive des politiques, panne du vector store et coûts LLM. Je les traite par secrets manager, antivirus, SSO/RBAC, backups, health checks et monitoring.

Que changeriez-vous pour passer à l'échelle?

Service d'embeddings séparé, Qdrant managé ou clusterisé, workers horizontaux, Redis partagé, PostgreSQL pour l'état durable, cache de retrieval, observabilité distribuée et tests de charge.

Comment expliquer les chiffres?

Je précise le snapshot, la taille du corpus et le type de questions. Je compare les pipelines et explique le coût en latence plutôt que de présenter uniquement le meilleur score.

Phrase finale forte

> *Ce projet montre que je sais relier recherche d'information, génération, outils structurés et exploitation logicielle dans un système mesurable et vérifiable.*

Annexe. Checklist portfolio

- Le README explique le problème, l'architecture, les commandes et les limites.
- Une capture montre une question et les sources [S1], [S2].
- Une seconde capture montre l'upload et l'indexation.
- Le dépôt ne contient pas .env, API keys, uploads ou bases privées.
- Les tests passent: pytest et Ruff.
- Le lancement Docker est documenté et reproductible.
- Le projet GitHub contient une licence et un workflow CI.
- Le CV renvoie vers GitHub et, si possible, une vidéo de démonstration.

Commandes portfolio

```
git status
git add README.md .gitignore .env.example .dockerignore Dockerfile docker-compose.yml
git add pyproject.toml requirements.txt LICENSE configs docs frontend frontend_web
git add scripts src tests .github
git commit -m "Document Enterprise RAG platform"
git push
```

Note: les résultats de benchmark présents dans le dépôt correspondent à un snapshot antérieur du corpus. Dans la session locale la plus récente, l'index validé contenait 639 documents et 1 926 chunks.